



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Migración de FPPC: Alto Rendimiento para Verificación de Tipos en GQL

INFORME FINAL DE CC6908 PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

Felipe Ignacio Avendaño Araya

MODALIDAD:
Memoria

PROFESOR GUÍA:
Matías Toro

SANTIAGO DE CHILE
2025

1. Introducción

La complejidad de los datos interconectados en dominios como redes sociales, bioinformática, sistemas de recomendación y detección de fraudes ha establecido a los grafos como modelo fundamental para representar relaciones. Los grafos de propiedades, que permiten atribuir metadatos tanto a los nodos como a las relaciones mediante pares clave-valor, han demostrado ser particularmente adecuados para modelar dominios complejos. Este reconocimiento motivó esfuerzos de estandarización que resultaron en la publicación de Graph Query Language (GQL) como estándar ISO/IEC 39075:2024 [9], destinado a unificar el ecosistema de consultas para grafos de propiedades.

La ausencia de un sistema de verificación estática de tipos en GQL representa una limitación significativa que afecta la experiencia del desarrollador y la confiabilidad de las aplicaciones. Consultas semánticamente incorrectas que podrían detectarse de manera estática solo se identifican durante la ejecución, generando errores tardíos, resultados vacíos sin retroalimentación adecuada y ciclos de depuración innecesarios. Este problema es más evidente en entornos con grandes volúmenes de datos, donde ejecutar consultas inválidas consume recursos considerables.

Para abordar esta limitación, Flexible Path Pattern Calculus (FPPC) [21] surge como una propuesta académica que extiende GQL con un sistema de tipos gradual, demostrando la viabilidad de detectar patrones inválidos estaticamente. Sin embargo, su implementación inicial en Python, aunque funcional, presenta limitaciones de rendimiento que afectan su aplicabilidad en escenarios productivos.

Esta propuesta aborda esta brecha mediante la migración del núcleo de FPPC a un lenguaje compilado de alto rendimiento. El objetivo es desarrollar una implementación que preserve las capacidades de verificación estática de tipos mientras alcanza el desempeño necesario para entornos productivos. El trabajo busca validar el valor práctico de integrar verificación estática en el flujo de trabajo con GQL, lo que podría facilitar la adopción del estándar en aplicaciones que requieren mayor robustez.

2. Estado del Arte

Un *property graph* o grafo de propiedades es un modelo de datos que extiende el concepto básico de grafo al permitir que tanto los nodos (vértices) como las relaciones (aristas) contengan atributos en forma de pares clave-valor. Esta característica fundamental diferencia este modelo de otros tipos de grafos, ya que enriquece la semántica de los elementos gráficos con metadatos específicos. Por ejemplo, un nodo que representa a una persona puede tener propiedades como **nombre**, **edad** y **ciudad**, mientras que una relación de amistad entre dos personas puede tener propiedades como **fecha_de_inicio** y **nivel_de_confianza**.

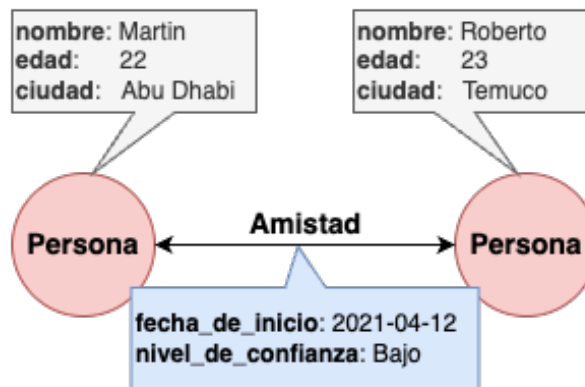


Figura 1: Ejemplo de grafo de propiedades.

La relevancia de este modelo radica en su capacidad para representar dominios complejos de manera natural, donde las entidades y sus conexiones portan información intrínseca que es crucial para el análisis. En el contexto de bases de datos, los grafos de propiedades pueden implementarse de forma nativa o como vistas sobre datos relacionales, lo que ha motivado el desarrollo de lenguajes de consulta especializados.

En este sentido, Neo4j emergió como pionero al introducir el lenguaje Cypher [16], que implementa la consulta de grafos mediante una sintaxis intuitiva basada en arte ASCII. Expresiones como `(a:Person)-[:Likes]->(b:Movie)` permiten describir patrones complejos de manera declarativa, haciendo las consultas más accesibles para desarrolladores. Sin embargo, la proliferación de lenguajes propietarios creó fragmentación en el ecosistema, impulsando la necesidad de un estándar unificado.

Esta necesidad fue abordada inicialmente con SQL/PGQ (Property Graph Query) [8], una extensión del estándar SQL que permite consultar grafos de propiedades desde bases de datos relacionales. Posteriormente, el comité ISO/IEC JTC 1/SC 32 [7] desarrolló el lenguaje GQL bajo el estándar ISO/IEC 39075:2024, que combina características de Cypher y SQL/PGQ en un estándar unificado. Sin embargo, al carecer de un sistema de verificación de tipos estático, GQL permite que consultas semánticamente incorrectas se ejecuten hasta completarse, retornando resultados vacíos sin proporcionar advertencias al usuario. La estandarización de GQL representa un avance significativo, pero esta limitación en su diseño presenta desafíos importantes para su adopción práctica.

En relación a lo anterior, MillenniumDB [14], desarrollado por el Millennium Institute for Foundational Research on Data, constituye una implementación del estándar GQL. Este sistema ofrece un *runtime* eficiente y soporta múltiples modelos de grafos, incluyendo grafos de propiedades. No obstante, MillenniumDB hereda la limitación del estándar al carecer de un sistema de verificación de tipos estático, lo que significa que consultas semánticamente incorrectas solo se detectan durante la ejecución, generando errores inesperados y resultados vacíos sin retroalimentación adecuada.

Por otra parte, en el ámbito académico, GPC (Graph Pattern Calculus) [5] presenta un modelo formal para patrones de grafos, pero se limita a un sistema de tipos de baja granularidad que no realiza seguimiento de etiquetas o tipos de propiedades específicos, y por lo tanto no puede prevenir todos los errores de ejecución o resultados inherentemente vacíos.

Para superar estas limitaciones, se desarrolló FPPC, un lenguaje que extiende GQL con un sistema de tipos gradual. Este sistema puede identificar patrones inválidos estaticamente, detectando inconsistencias como la comparación de tipos incompatibles. FPPC introduce una extensión sintáctica (traducible a GQL, aunque de forma verbosa) que permite filtrar elementos de patrones basándose en expresiones de propiedades, facilitando el filtrado directo de nodos y aristas considerando tanto etiquetas como tipos de propiedades. Por ejemplo, el patrón $(y : \star \{\text{name} : \text{str}, \star\})$ filtra todos los nodos que tienen cualquier etiqueta (e.g., **Person**, **Student**, etc.) y una propiedad **name** de tipo **string**.

Para profundizar en la estructura formal de FPPC, a continuación se presenta la gramática BNF (Backus-Naur Form) del núcleo del lenguaje, extraída directamente de la publicación original [21]. Para un análisis teórico exhaustivo de todas las reglas y su semántica, se remite al lector a dicha fuente.

Base Types	$\iota ::= \mathbb{Z} \mid \mathbb{B} \mid \text{String}$
Simple Types	$\tau ::= \iota \mid \star \mid \tau + \tau \mid \perp$
Attributes	$a \in \{\text{name}, \text{status}, \text{a}, \text{b}, \dots\}$
Property Types	$R ::= \{\overline{a_i} : \overline{\tau_i^i}\} \mid \{\overline{a_i} : \overline{\tau_i^i}, \star\} \mid \perp$
Labels	$l \in \{\text{Person}, \text{Student}, \text{A}, \text{B}, \dots\}$
Label Types	$\ell ::= l \mid \star \mid \ell \& \ell \mid \ell + \ell \mid \varepsilon$
Descriptor Types	$L ::= \ell R$
Constants	$c ::= z \mid s \mid b$
Expressions	$t ::= c \mid t \text{ bop } t \mid \text{uop } t \mid x.a \mid t \text{ as } \tau \mid t \text{ is } \tau$
Optional Variables	$x? ::= x \mid _$
Descriptors	$d ::= x? : L$
Directions	$\Longleftrightarrow ::= \Leftarrow \mid \rightarrow \mid \sim \mid -$
Path Patterns	$\pi ::= (d) \mid \xrightarrow{d} \mid \pi + \pi \mid \pi \pi \mid \pi_{(t)} \mid \pi^{l..u}$

Figura 2: Definición formal de la sintaxis de FPPC

En este formalismo, la entidad central es el patrón de camino o *path pattern* (π). La gramática define sus variantes estructurales, incluyendo patrones de nodo ((d)), aristas ($\overset{d}{\longleftrightarrow}$), uniones de caminos ($\pi + \pi$), entre otros. Cabe destacar la notación $\pi_{\langle t \rangle}$, que actúa como un filtro sobre el camino basado en la expresión t , análogo a la operación de selección en álgebra relacional o a las cláusulas **WHERE** en lenguajes de consulta estándar.

El aspecto más innovador de esta propuesta reside en los *property types* (R), que permiten declarar explícitamente las propiedades y sus tipos, siendo clave para el *gradual typing* y la detección de inconsistencias. Si bien el formalismo contempla tres variantes, nos enfocamos en las dos primeras por su relevancia en el modelado de datos:

- La primera variante, $\{\overline{a_i} : \overline{\tau_i^i}\}$, representa tipos de propiedades cerrados, lo que implica que el patrón debe tener exactamente las propiedades declaradas y ninguna otra.
- La segunda variante, $\{\overline{a_i} : \overline{\tau_i^i}, \star\}$, denota tipos de propiedades abiertos, permitiendo la existencia de propiedades adicionales no declaradas explícitamente. Nótese que $\star$ es análogo al *wildcard* ($*$) o el tipo **any** en varios lenguajes de programación, significando “cualquier tipo” en la mayoría de los contextos.

Es importante recalcar que FPPC es una extensión de GQL, y por lo tanto, no todas sus expresiones serán válidas en el estándar GQL actual. FPPC propone esta extensión específicamente para los patrones de camino (*path patterns*); si bien estos no constituyen la totalidad del lenguaje GQL, son su núcleo fundamental y el lugar crítico donde la verificación de tipos propuesta en el *paper* aporta su valor diferencial. Dado el alcance actual de este formalismo, se proyecta que el lenguaje continúe evolucionando para alinearse integralmente con el estándar GQL, ya sea mediante la adaptación de su sintaxis o la incorporación de futuras sentencias.

Actualmente, las reglas de validación y la semántica de este lenguaje se encuentran materializadas en un **prototipo de referencia implementado en Python** [21]. Esta herramienta implementa la lógica de verificación de tipos (realizando ciertos ajustes sobre la sintaxis concreta) y es capaz de entregar diagnósticos precisos al usuario. Para ilustrar las capacidades del sistema de verificación de tipos, consideremos la siguiente consulta en la implementación de FPPC:

```
(x: {status: bool} WHERE x.status > 0) - [:Author WHERE y.foo] ->
```

Esta consulta es sintácticamente válida, pero contiene tanto una inconsistencia de tipos (la comparación de un valor booleano con un entero) como una referencia a una variable no vinculada. El verificador de tipos del prototipo es capaz de detectar estos problemas estáticamente y notificar al usuario.

A pesar de esta validez funcional, su implementación actual presenta limitaciones de rendimiento inherentes a la naturaleza interpretada del lenguaje en el que está escrito, lo que restringe su aplicabilidad en entornos productivos con grandes volúmenes de datos. Esta situación evidencia la necesidad de una implementación que combine las capacidades de verificación estática de tipos de FPPC con el rendimiento necesario para entornos productivos, lo que justifica el desarrollo de una versión optimizada en un lenguaje compilado de alto rendimiento.

3. Objetivos

Objetivo General

Desarrollar una implementación de alto rendimiento del sistema de verificación de tipos gradual FPPC para GQL, funcionalmente equivalente al prototipo original en Python pero significativamente más eficiente.

Objetivos Específicos

1. Garantizar la compatibilidad funcional con el prototipo FPPC en la identificación de inconsistencias de tipos y patrones de consulta inválidos.
2. Implementar el sistema de verificación de tipos en un lenguaje compilado de alto rendimiento.
3. Evaluar el impacto del sistema de verificación de tipos de la re-implementación de FPPC.
4. Elaborar documentación técnica integral que facilite el uso, extensión e integración del sistema.

Evaluación

La evaluación del trabajo se realizará mediante un enfoque metodológico que permita medir cuantitativamente el cumplimiento de cada objetivo específico. Para el primer objetivo (equivalencia funcional), se utilizará el conjunto de pruebas existente del prototipo FPPC original, que incluye casos de consultas válidas e inválidas con esquemas de diferente complejidad. Se ejecutarán estas pruebas en ambas implementaciones y se compararán sistemáticamente los resultados para verificar la detección idéntica de inconsistencias de tipos y patrones vacíos.

Para evaluar el segundo objetivo, se implementará un *benchmarking* que medirá el porcentaje de mejora temporal mediante el uso de herramientas estándar de medición de desempeño. El análisis empleará esquemas de grafos de diferentes escalas y consultas de complejidad variable, estableciendo una línea base con la implementación original en Python y comparándola con la nueva implementación.

Para el tercer objetivo, se utilizará esta misma métrica de porcentaje de mejora temporal junto con los mismos esquemas de grafos de diferentes escalas, evaluando específicamente consultas con inconsistencias semánticas o que siempre produzcan resultados vacíos. La comparación se realizará entre la reimplementación de FPPC e implementaciones de GQL desarrolladas en lenguajes de rendimiento similar, lo que permitirá aislar el beneficio de la verificación estática de tipos mediante una comparación justa entre sistemas tecnológicamente equivalentes.

4. Solución Propuesta

Para lograr los objetivos planteados, se propone la migración del sistema de verificación de tipos FPPC desde Python hacia el lenguaje de programación Rust. Esta elección se fundamenta en que Rust es un lenguaje compilado que genera código nativo de alto rendimiento, ofrece seguridad de memoria en tiempo de compilación sin necesidad de un recolector de basura, y tiene un modelo de concurrencia seguro que previene condiciones de carrera. La factibilidad de esta aproximación está respaldada por proyectos exitosos como **Pyrefly** [13] y **ty** [2] (*type checkers* para Python escritos en Rust), que demuestran cómo Rust puede mejorar significativamente el rendimiento de herramientas de análisis estático sin comprometer la confiabilidad.

La implementación utilizará las siguientes herramientas y enfoques:

1. **Rust como lenguaje objetivo:** Se aprovecharán sus garantías de seguridad de memoria y su ecosistema de alto rendimiento. La existencia de proyectos como Pyrefly y ty evidencian que Rust es una elección técnica sólida para implementar sistemas de tipos eficientes.
2. **Librerías específicas para la implementación:**
 - **Parsing:** Para el *parser*, se utilizará el *crate* (librería/binario en Rust) `lalrpop` [11] (generador de *parsers* LR [1]), aprovechando las capacidades de *parsing* de alto rendimiento de Rust.
 - **Estructuras de datos:** Se utilizarán estructuras eficientes de la librería estándar de Rust.
 - **Testing:** Para garantizar la corrección, se utilizará el framework de *testing* integrado de Rust, complementado con *crates* como `proptest` [18] para pruebas basadas en propiedades (generación de inputs aleatorios) y `test-log` [15] para manejar la salida de logs durante las pruebas.
3. **Implementación del sistema de tipos:** Las reglas de verificación se implementarán utilizando el sistema de tipos de Rust, siguiendo sus principios de *ownership* y *borrowing* [20] para garantizar seguridad de memoria.
4. **Evaluación de rendimiento:**
 - **Implementación en Rust:** Se utilizará `Criterion.rs` [6] para un *benchmarking* preciso y estadístico del tiempo.
 - **Implementación Original (Python):** Se utilizará el módulo estándar `timeit` [19] para la medición del tiempo.
 - **Esquemas:** La evaluación utilizará esquemas de grafos de diferentes tamaños (pequeño, mediano y grande) obtenidos o inferidos a partir de datos reales disponibles públicamente, junto con consultas de complejidad variable. En caso de que no se disponga de datasets en formato GQL, se transformarán datasets existentes de *property graphs* al estándar GQL mediante procesos de conversión apropiados.
 - **MillenniumDB:** Para evaluar el valor añadido del sistema de verificación de tipos, se contrastará el flujo de trabajo con FPPC (que permite advertir que una consulta será vacía o contiene errores sin necesidad de ejecutarla) contra la ejecución directa en MillenniumDB (que detecta estos problemas durante la ejecución). Para asegurar mediciones precisas en MillenniumDB, se utilizarán herramientas

de medición de tiempo como `std::chrono` [4] en C++. Esta comparación es técnicamente válida ya que ambas implementaciones utilizan lenguajes compilados de alto rendimiento (Rust para FPPC y C++ para MillenniumDB), lo que permite aislar el beneficio específico de la verificación estática sin sesgos por diferencias tecnológicas fundamentales.

5. Plan de Trabajo

1. **Análisis del Prototipo** (Semana 1):
 - (a) Estudiar a fondo la implementación actual en Python.
 - (b) Extraer y comprender la *suite* de pruebas existente.
 - (c) Definir un conjunto inicial de pruebas de regresión (basadas en el prototipo) que servirán como métrica de equivalencia funcional del sistema de verificación de tipos.
2. **Aprendizaje de Rust Dirigido por el Proyecto** (Semanas 2-4):
 - (a) Aprender los fundamentos de Rust necesarios para el *type checker* (*ownership*, *enums*, *traits*, manejo de errores, etc.).
 - (b) Practicar escribiendo *tests* unitarios en Rust usando las librerías especificadas.
 - (c) Familiarizarse con las librerías clave (`lalrpop`, `nom`, etc.) mediante la creación de prototipos simples.
3. **Implementación Test-Driven del Parser** (Semanas 5-11):
 - (a) Para una característica de GQL/FPPC a la vez, escribir primero *tests* que definan el comportamiento esperado.
 - (b) Implementar el *parser* usando `lalrpop` para que pase los *tests*.
 - (c) Refactorizar y repetir hasta cubrir la gramática completa.
4. **Implementación Test-Driven del Type Checker** (Semanas 12-16):
 - (a) Para cada regla de verificación de tipos (ej: unificación, subtipado, inferencia), escribir *tests* que capturen su comportamiento, incluidos los casos de error y de tipado inconsistente.
 - (b) Implementar la lógica de verificación de tipos en Rust para hacer pasar los *tests*.
5. **Integración y Pruebas** (Semanas 17-18):
 - (a) Integrar los componentes (*parser* y *type checker*) en un flujo de trabajo unificado.
 - (b) Ejecutar la *suite* completa de pruebas de regresión (ref. primer paso) para validar la equivalencia funcional con el prototipo Python.
6. **Evaluación de Rendimiento** (Semanas 19-22):
 - (a) Identificar y seleccionar esquemas de grafos de diferentes escalas (pequeños, medianos, grandes) a partir de conjuntos de datos públicos y repositorios especializados que representen casos de uso reales. Como plan alternativo, se transformarán datasets de Neo4j [17] al estándar GQL.
 - (b) Diseñar e implementar *benchmarks* para medir el rendimiento del sistema de verificación de tipos en consultas y esquemas de diversa complejidad.
 - (c) Ejecutar los *benchmarks* en ambas implementaciones (Python y Rust) y documentar las mejoras.
 - (d) Ejecutar un subconjunto de los *benchmarks* existentes que contengan consultas semánticamente incorrectas (vacías o con errores de tipo), comparando el tiempo de detección entre FPPC (verificación estática) y MillenniumDB (detección durante ejecución).

7. **Documentación e Informe** (*Incrementalmente*: Semanas 8, 11, 16, 22, 23):

- (a) Documentar el código y la arquitectura de la nueva implementación.
- (b) Redactar el informe, detallando la metodología TDD (Test Driven Development), implementación, los resultados y las conclusiones.

En base a los pasos descritos anteriormente, se presenta a continuación el cronograma que organiza el trabajo de las **últimas 15 semanas** del proyecto, correspondientes al semestre del curso “Trabajo de Título” (CC6919).

		Semana														
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Parser TDD																
Type-Checker TDD																
Integración y Pruebas																
Evaluación de Rendimiento																
Documentación e Informe																

6. Trabajo Adelantado

En concordancia con las primeras fases definidas en el Plan de Trabajo, se realizó un avance sustancial centrado en establecer los cimientos del sistema en Rust. Este trabajo abarcó desde la disección del prototipo original hasta la implementación de un *parser* **funcional** [3] para un subconjunto del lenguaje, aplicando mejoras de ingeniería de software y calidad de código.

6.1. Análisis del Prototipo y Hallazgos

La inspección técnica del código fuente en Python reveló divergencias claves respecto a la especificación teórica y confirmó viabilidad de librerías seleccionadas:

- **Divergencias Sintácticas:** Se constató que el prototipo implementa una sintaxis concreta distinta a la notación formal del *paper* para optimizar la experiencia del usuario. El código incorpora “azúcar sintáctico” (como reducción de paréntesis, omisión y valores por defecto) y notaciones específicas. Entre estas últimas destacan el uso de `|` para uniones y la distinción entre `{...}` y `{{...}}` para declaraciones de propiedades abiertas y cerradas, respectivamente.
- **Identificación de Tests:** Se aislaron las *suites* de pruebas críticas dentro del repositorio. Se identificó `typechecker_test.py` como el estándar para la futura validación de equivalencia funcional, y se extrajeron los casos de `parser_test.py` para instrumentar el desarrollo actual bajo TDD.
- **Validación Incidental de Herramientas:** Se observó que el prototipo utiliza la librería Lark con el algoritmo LALR(1). Esto confirmó la idoneidad de `lalrpop` para Rust, ya que este genera *parsers* LR(1). Dado que LR(1) es un superconjunto de LALR(1) [1], esto garantiza teóricamente que la gramática del prototipo es compatible con la nueva herramienta, ofreciendo incluso mayor potencia para resolver conflictos sintácticos si fuera necesario.

6.2. Capacitación Técnica Dirigida

El aprendizaje del lenguaje se llevó a cabo siguiendo la metodología experimental del “Rust Book Experiment” [10] (Brown University). Se completaron los ejercicios prácticos y *quizzes* de los capítulos 1 al 7, 9 y 11, priorizando así la sintaxis básica, el dominio de la gestión de memoria (*ownership*, *borrowing*) y las herramientas nativas de *testing*. Respecto a la herramienta de *parsing*, la capacitación se basó en la ejecución práctica del tutorial oficial de `lalrpop` [12].

6.3. Implementación y Metodología TDD

Con los conocimientos adquiridos, se implementó el *parser* para un subconjunto de *path patterns*, que serían los patrones de nodo (Node Patterns) junto a cláusulas **WHERE**. Este subconjunto permite procesar patrones sintácticos como:

- Nodos Simples y con Variables:
(), (x)
- Nodos con *Labels*:
(x: **Person**), (x: **Author & Teacher**)
- Nodos con propiedades:
(x: {**name**: **str**, **status**: **bool**})
- Filtros Complejos:
(x: **Teacher** {**age**: **int**, **active**: **bool**} **WHERE** x.**age** >= 40 **AND** x.**active**)

Decisión de Diseño: Se decidió explícitamente mantener las modificaciones sintácticas del prototipo en la implementación de Rust, en lugar de revertir a la notación del *paper*. Esta decisión prioriza la consistencia con la herramienta existente.

Estructura del Proyecto: Para soportar esta implementación, se diseñó una arquitectura modular que separa claramente las definiciones del AST de las reglas gramaticales, facilitando la mantenibilidad y extensión futura (ver estructura de directorios a continuación).

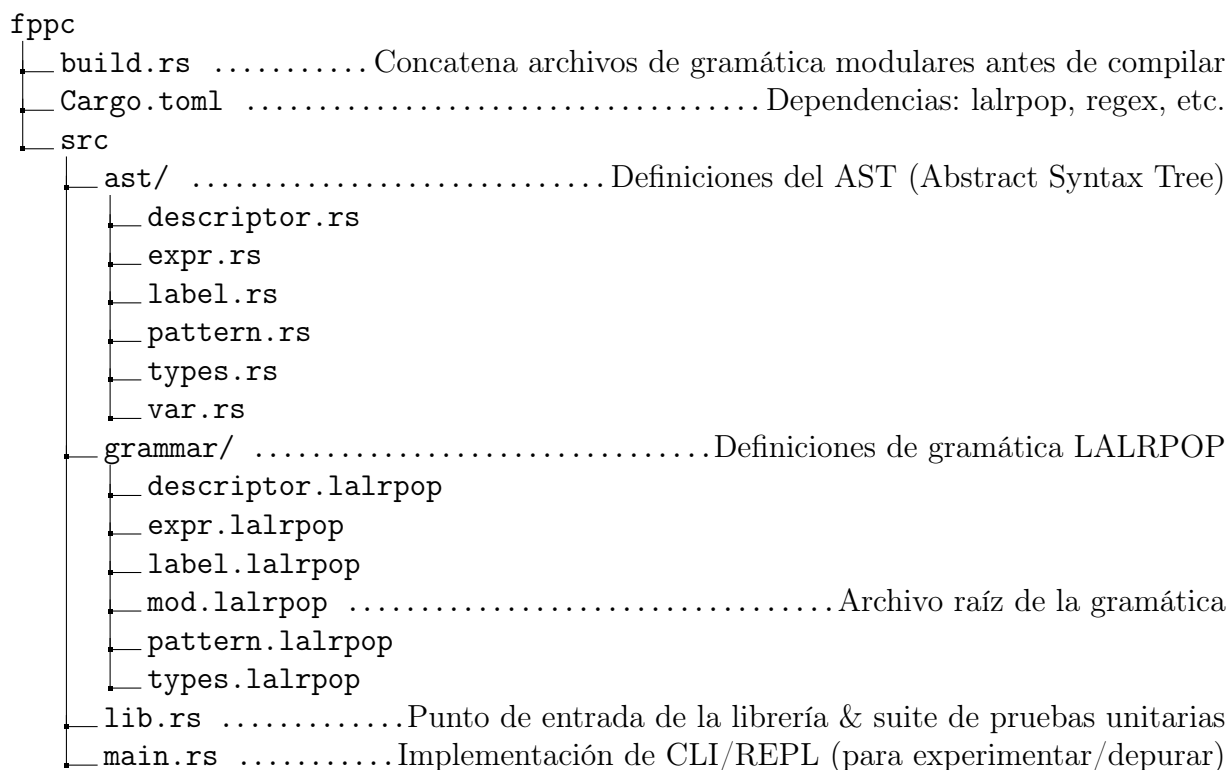


Figura 3: Estructura de directorios del proyecto.

En particular, el directorio `src/ast/` define los tipos de datos en Rust que constituyen los nodos del Árbol de Sintaxis Abstracta (AST). Por otro lado, el directorio `src/grammar/`

contiene las definiciones de la gramática utilizando LALRPOP, el cual emplea una sintaxis análoga a Rust para definir las reglas de producción y su correspondencia directa a las estructuras del AST.

Una innovación de esta implementación es la modularización de la gramática en múltiples archivos; esto es posible gracias al *script* `build.rs`, el cual concatena estos módulos en una única definición gramatical durante la compilación. Finalmente, `src/main.rs` implementa una interfaz de línea de comandos (CLI/REPL) para la depuración granular de reglas. A continuación se muestra un ejemplo de uso donde se valida un patrón de nodo:

```
> path (x: Person)
Valid: Node(Descriptor(Some(Var(x)), DescriptorType(Label(Person), Open({}))))
```

Metodología de Desarrollo (TDD): El desarrollo siguió una estrategia de Test Driven Development incremental. Inicialmente, se definieron unidades elementales AST (e.g., `Label`, `Descriptor`) y se crearon reglas de producción específicas para ellas. Estas unidades se validaron de forma aislada dentro del módulo de pruebas en `src/lib.rs` (siguiendo la convención de Rust de alojar pruebas unitarias junto al código, ejecutables mediante `cargo test`), utilizando fragmentos extraídos de los casos de `parser_test.py`.

Una vez que los componentes base fueron estables, se procedió a implementar el tipo objetivo de nivel superior `PathPattern`, concretamente sus variantes `Node` (para nodos) y `Filter` (para la aplicación de cláusulas `WHERE` sobre un patrón). En este punto, la estrategia de pruebas evolucionó: se eliminaron los *tests* individuales de los componentes elementales y se consolidó la validación en el *parser* de nivel superior, ejecutando directamente los casos de prueba completos del prototipo original. Esto garantizó que, dentro del alcance del subconjunto soportado, el sistema integrado fuese funcionalmente equivalente a la referencia en Python.

Un ejemplo representativo de estos tests consolidados se muestra a continuación:

```
1 mod tests {
2     use super::*;
3     // ...
4     #[test]
5     fn test_node_empty() {
6         let result = PathPatternParser::new().parse("()").unwrap();
7         let expected = PathPattern::Node(Descriptor {
8             variable: None,
9             descriptor_type: DescriptorType {
10                 label: LabelType::Star,
11                 properties: PropertyType::open(),
12             },
13         });
14         assert_eq!(result, expected);
15     }
16     // ...
17 }
```

6.3.1. Mejoras Arquitectónicas y Corrección de Errores

La naturaleza estática de Rust y el uso de `lalrpop` permitieron mejoras estructurales significativas respecto al diseño original:

- **Seguridad de Tipos:** La principal mejora respecto al prototipo radica en la naturaleza del lenguaje: a diferencia de Python, que posee un tipado dinámico, Rust impone un sistema de tipos estático y fuerte.

Adicionalmente, para definir la gran mayoría de los elementos del AST se utilizaron `enums` (Tipos de Datos Algebraicos), lo que obliga a un manejo exhaustivo de casos y previene estados inválidos. A modo de ejemplo, y para ilustrar este patrón de diseño recurrente en el proyecto, considérese la definición de `LabelType`:

```
1 #[derive(PartialEq, Clone, Default)]
2 pub enum LabelType {
3     Label(String),
4     #[default]
5     Star,
6     And(Box<LabelType>, Box<LabelType>),
7     Or(Box<LabelType>, Box<LabelType>),
8 }
```

La declaración comienza con `pub enum LabelType`, definiendo un tipo enumerado público accesible por los módulos de gramática. En Rust, una instancia de `LabelType` puede adoptar únicamente una de sus variantes a la vez. Una característica fundamental de este patrón es que las variantes pueden contener datos heterogéneos asociados (e.g., `Label` porta un `String`, mientras que `Star` es unitario), permitiendo modelar la estructura polimórfica del árbol de forma precisa.

Sobre esta estructura, el atributo `#[derive(...)]` utiliza macros procedimentales para inyectar código automáticamente, implementando *traits* esenciales sin *boilerplate*:

- `PartialEq`: Habilita la igualdad estructural, permitiendo comparar si dos árboles AST son idénticos (crucial para las aserciones `assert_eq!` en los tests unitarios).
- `Clone`: Permite realizar copias profundas (*deep copies*) de la estructura recursiva cuando es necesario bifurcar el flujo de datos.
- `Default`: Genera un constructor por defecto, configurado explícitamente mediante la anotación `#[default]` sobre la variante `Star`.

Finalmente, el uso del puntero inteligente `Box<...>` en las variantes recursivas (`And`, `Or`) es una necesidad arquitectónica impuesta por el modelo de memoria de Rust. Dado que el compilador debe conocer el tamaño exacto (en bytes) de todo tipo ubicado en el *stack*, una estructura recursiva directa generaría un tipo de tamaño infinito. `Box` resuelve esto moviendo los nodos hijos al *heap* y almacenando en el `enum` únicamente un puntero de tamaño fijo, garantizando así que el tipo sea compilable.

- **Simplificación Declarativa:** El uso de *macros* de `lalrpop` permitió definir la precedencia de operadores y la asociatividad de forma declarativa en la gramática. Esto eliminó la necesidad de definir reglas de producción intermedias (comunes en *parsers* para manejar precedencia) y algoritmos de post-procesamiento, resultando en una construcción más directa del AST.

Por ejemplo, la definición de la precedencia en la gramática de etiquetas (*labels*) se reduce a anotaciones concisas, en contraste con la estructura jerárquica explícita requerida en Lark.

Rust (LALRPOP)

```
1 pub LabelType: LabelType = {
2     #[precedence(level="2")] #[assoc(side="left")]
3     <l:LabelType> "|" <r:LabelType> => LabelType::or(l, r),
4
5     #[precedence(level="1")] #[assoc(side="left")]
6     <l:LabelType> "&" <r:LabelType> => LabelType::and(l, r),
7     // ...
8 }
```

Python (Lark)

```
1 gql_grammar = r"""
2 ...
3 ?label_pattern: label_term | label_disjunction
4 label_disjunction: label_pattern "|" label_term
5 ?label_term: label_factor | label_conjunction
6 label_conjunction: label_term "&" label_factor
7 ?label_factor: label_primary | label_negation
8 label_negation: "!" label_primary
9 ?label_primary: label_name | "(" label_pattern ")"
10 ...
11 """
```

Figura 4: Comparación de la definición de precedencia. LALRPOP (arriba) permite anotaciones directas, mientras que Lark (abajo) requiere una jerarquía de reglas explícita.

Además, el análisis estricto de la gramática permitió identificar y eliminar reglas de producción “muertas” (definidas en el prototipo pero nunca referenciadas).

- **Corrección de AST:** Se identificó un *bug* estructural donde los literales de tipo en expresiones (e.g., `x IS int`) carecían de definición en el AST del prototipo (se sospecha que seguía funcionando gracias a *duck typing*). En Rust, esto se corrigió formalmente definiendo `TypeLiteral` como una variante explícita.

Referencias

- [1] Aho, Alfred V., Monica S. Lam, Ravi Sethi y Jeffrey D. Ullman: *Compilers: Principles, Techniques, and Tools*. Pearson Education Limited, 2ª edición, 2014.
- [2] Astral Software Inc.: *ty: An extremely fast Python type checker (Documentation)*. <https://docs.astral.sh/ty/>.
- [3] Avendaño, Felipe: *FPPC-rs: Flexible Path Pattern Calculus in Rust*. <https://github.com/Felipe705x/fppc>, 2025.
- [4] *std::chrono reference*. <https://cplusplus.com/reference/chrono/>.
- [5] Francis, Nadime, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova y Domagoj Vrgoc: *GPC: A Pattern Calculus for Property Graphs*. En *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '23)*, páginas 241–250. ACM, 2023. <https://dl.acm.org/doi/10.1145/3584372.3588662>.
- [6] Heisler, Brook: *Criterion.rs Documentation*. <https://bheisler.github.io/criterion.rs/book/>.
- [7] International Organization for Standardization (ISO): *ISO/IEC JTC 1/SC 32: Data management and interchange*. <https://www.iso.org/committee/45342.html>.
- [8] International Organization for Standardization (ISO) y International Electrotechnical Commission (IEC): *Information technology — Database languages SQL — Part 16: Property Graph Queries (SQL/PGQ)*. Informe técnico ISO/IEC 9075-16:2023, ISO/IEC JTC 1/SC 32, Junio 2023. <https://www.iso.org/standard/79473.html>, Edición 1. ICS 35.060.
- [9] International Organization for Standardization (ISO) y International Electrotechnical Commission (IEC): *Information technology — Database languages — GQL*. Informe técnico ISO/IEC 39075:2024, ISO/IEC JTC 1/SC 32, Abril 2024. <https://www.iso.org/obp/ui/en/#iso:std:76120:en>, Edition 1. ICS 35.060.
- [10] Klabnik, Steve, Carol Nichols y Will Crichton: *The Rust Programming Language: Experiment Introduction*. <https://rust-book.cs.brown.edu/>, 2023.
- [11] Matsakis, Nicholas, et al.: *LALRPOP: LR(1) parser generator for Rust*. <https://github.com/lalrpop/lalrpop>. Official GitHub repository.
- [12] Matsakis, Nicholas, et al.: *LALRPOP Tutorial*. <https://lalrpop.github.io/lalrpop/tutorial/index.html>.
- [13] Meta Platforms, Inc.: *Pyrefly: A fast Python type checker and language server*. <https://pyrefly.org/>.
- [14] Millennium Institute for Foundational Research on Data (IMFD): *MillenniumDB*.

<https://github.com/MillenniumDB/MillenniumDB>. Official GitHub repository.

- [15] Müller, Daniel: *test-log* - *crates.io: Rust Package Registry*. <https://crates.io/crates/test-log>.
- [16] Neo4j, Inc.: *Cypher Manual — Introduction*. <https://neo4j.com/docs/cypher-manual/current/introduction/>.
- [17] Neo4j, Inc.: *Neo4j Getting Started — Example Data*. <https://neo4j.com/docs/getting-started/appendix/example-data/>.
- [18] *Introduction - Proptest*. <https://altsysrq.github.io/proptest-book/>.
- [19] Python Software Foundation: *timeit — Measure execution time of small code snippets*. <https://docs.python.org/3/library/timeit.html>.
- [20] *Understanding Ownership*. <https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html>. The Rust Programming Language, cap. 4.
- [21] Ye, Wenjia, Matías Toro, Tomás Díaz, Bruno C. d. S. Oliveira, Manuel Rigger, Claudio Gutierrez y Domagoj Vrgoč: *Flexible and Expressive Typed Path Patterns for GQL*. En *Proceedings of the ACM on Programming Languages (Proc. ACM Program. Lang.)*, volumen 9 de *OOPSLA2*, páginas 1–27. ACM, Octubre 2025. <https://doi.org/10.1145/3763061>, Preprint at <https://pleiad.dcc.uchile.cl/papers/2025/yeA1-oopsla2025.pdf>.